

Computer Generation of Hypergeometric Random Variates†

VORATAS KACHITVICHYANUKUL

Department of Industrial and Management Engineering, The University of Iowa, Iowa City, Iowa 52242, U.S.A.

and

BRUCE SCHMEISER

School of Industrial Engineering, Purdue University, West Lafayette, Indiana 47907

(Received July 6, 1984; in final form April 9, 1985)

The paper presents an exact, uniformly fast algorithm for generating random variates from the hypergeometric distribution. The overall algorithm framework is acceptance/rejection and is implemented via composition. Three subdensities are used, one is uniform and the other two are exponential. The algorithm is compared with algorithms based on sampling without replacement, inversion, and aliasing. A comprehensive survey of existing algorithms is also given.

KEY WORDS: Random variate generation, hypergeometric distribution, sampling without replacement.

1. INTRODUCTION

The focus of this report is on the generation of variates from the hypergeometric distribution, which is defined as the distribution of a

†Research Report Number 83-1, Department of Industrial and Management Engineering, The University of Iowa.

discrete random variable X for which $0 < k < n_1 + n_2$ and

$$f_H(x) = \frac{\binom{n_1}{x} \binom{n_2}{k-x}}{\binom{n_1+n_2}{k}} \quad 5 \max(0, k-n_2) \leq x \leq \min(k, n_1)$$

$$= 0 \quad \text{elsewhere}$$

The hypergeometric distribution is widely applied in the area of production and quality control, Bayesian estimation and hypothesis testing (see Lieberman and Owen, 1961). It applies to a finite population of size $n = n_1 + n_2$, n_1 of which have a particular attribute and n_2 of which do not. When a random sample of size k is drawn from such population, the number of items with the attribute is hypergeometrically distributed with parameters n_1 , n_2 , and k .

A review of existing algorithms is presented in Section 2. The first uniformly fast algorithm *H2PE* is developed in Section 3 and the computational results are given in Section 4. Results concerning the validity of the algorithm are given in Appendix I.

2. LITERATURE REVIEW

Schmeiser (1980) surveys random variate generation in general. Only algorithms for the hypergeometric distribution are considered here. Of these algorithms only two, both based on basic properties of the distribution, are available for the generation of the hypergeometric random variate. The first algorithm is based on the concept of sampling without replacement; i.e., the procedure is based on a sequence of trials in which the probability of success depends on the number of previous successes. This algorithm, denoted here by *HBU*, is equivalent to algorithms by Fishman (1973) and McGrath and Irving (1973).

Algorithm *HBU* (n_1, n_2, k)

0. Set $n \leftarrow n_1 + n_2$, $T \leftarrow n$, $T_1 \leftarrow n_1$

$x \leftarrow 0$, $J \leftarrow 0$

1. Repeat

$J \leftarrow J + 1$

Generate $u \sim U(0, 1)$

If $u \leq T_1/T$

then $x \leftarrow x + 1$

If $x = n_1$ return x

$T_1 \leftarrow T_1 - 1$

End if

End if

$T \leftarrow T - 1$

Until $J = k$

Return x .

This algorithm executes with time proportional to $\min(k, n_1)$ and the efficiency relies mainly on the speed of the uniform $(0, 1)$ generator. When k is greater than $(n_1 + n_2)/2$, the efficiency of this algorithm can be improved by replacing k by $n_1 + n_2 - k$ and returning $n_1 - x$ as the desired variate.

The second algorithm is based on the inverse transformation method and is described, e.g., in Fishman (1978). The implementation is based on the recursive formula

$$f_H(x+1) = \frac{(n_1 - x)(k - x)}{(x+1)(n_2 - k + 1 + x)} f_H(x)$$

beginning with

$$f_H(0) = \frac{n_2!(n-k)!}{n!(n_2-k)!} \quad \text{if } k < n_2$$

or with

$$f_H(k-n_2) = \frac{n_1!k!}{(k-n_2)!n!}$$

if $k > n_2$.

Algorithm $HIN(n_1, n_2, k)$

0. If $k < n_2$ set

$$p \leftarrow \frac{n_2!(n_1+n_2-k)!}{(n_1+n_2)!(n_2-k)!} \quad \text{and} \quad x=0$$

Otherwise set

$$p \leftarrow \frac{n_1!k!}{(k-n_2)!(n_1+n_2)!} \quad \text{and} \quad x=k-n_2$$

1. Generate $u \sim U(0, 1)$

If $u \leq p$ return x

Repeat

$$u \leftarrow u - p$$

$$p \leftarrow p(n_1-x)(k-x)/(x+1)(n_2-k+1+x)$$

$$x \leftarrow x + 1$$

Until $n \leq p$

Return x .

The speed of this method can be improved by selecting parameter values to minimize the number of comparisons. Let n_1^* , n_2^* and k^* denote the original parameters. Then the appropriate choices of parameters are:

	$k^* \leq (n_1^* + n_2^*)/2$	$k^* > (n_1^* + n_2^*)/2$
$n_1^* \leq n_2^*$	$k = k^*$ $n_1 = n_1^*$ $n_2 = n_2^*$ return x	$k = n_1^* + n_2^* - k^*$ $n_1 = n_1^*$ $n_2 = n_2^*$ return $n_1^* - x$
$n_1^* > n_2^*$	$k = k^*$ $n_1 = n_2^*$ $n_2 = n_1^*$ return $k^* - x$	$k = n_1^* + n_2^* - k^*$ $n_2 = n_1^*$ $n_1 = n_2^*$ return $k^* - n_2^* + x$

This algorithm executes with time proportional to the mean $n_1 k / (n_1 + n_2)$, which is more efficient than *HBU* but develops numerical problems when the mean is large, due to the accumulation of truncation errors. In addition, the value of p in Step 0 of *HIN* can underflow when the population size n is only moderately large. For example, in the worst case $n_1 = n_2$ and $k = (n_1 + n_2)/2$, $p = 7.2544 \times 10^{-12}$ when the mean is 10. Hence, straightforward implementation of the inverse transformation does not lead to satisfactory algorithm for hypergeometric distribution.

The two drawbacks discussed above can be eliminated by: (a) rescale p with a large constant, and (b) use better search algorithms such as bucket-binary search by Fox (1978), and index search by Ahrens and Kohrt (1981). These remedies improve marginal execution time at the expense of increasing setup time and memory requirements.

The alias method, see Walker (1977) or Kronmal and Peterson (1979), a composition approach, is a source of efficient algorithms for generating random variates from discrete distributions with a finite number of mass points. While no hypergeometric algorithm based on this method has been published, the method can be easily implemented for the hypergeometric distribution as follows:

Algorithm *HALIAS* (n_1, n_2, k)

Setup

1. Set $I_L \leftarrow \max\{0, k - n_2\}$, $I_U \leftarrow \min\{k, n_2\}$, $m \leftarrow I_U - I_L + 1$.
2. Calculate the hypergeometric probabilities for all mass points $I_L, I_L + 1, \dots, I_U$ and store in an array in position 1 to m .

3. Setup the alias table $A(\cdot)$ and the table of its cutoff values $f(\cdot)$, each also stores in an array (see Kronmal and Peterson [1979]). To generate one hypergeometric random variate:
4. Generate $u \sim U(0, 1)$, set $u \leftarrow u(m+1)$, $i \leftarrow [u] + 1$.
5. Set $x \leftarrow i$. If $u > F(i)$, set $x \leftarrow A(i)$.
6. Return $x \leftarrow x + I_L - 1$.

The marginal execution time of the alias algorithm is uniformly fast but the setup time and memory requirements are proportional to m . Hence, *HALIAS* is not suitable for large m or when the parameters change often. In addition, the rescaling of probabilities in Step 2 is required to avoid underflow when m is large.

3. ALGORITHM H2PE

A new algorithm, *H2PE*, is developed in this section. In addition to avoiding the numerical problems of *HIN* and *HALIAS*, *H2PE* is uniformly fast; i.e., the execution time is bounded over the set of parameter values for which the algorithm is intended. *H2PE* is intended for use when $M - \max(0, k - n_2) \geq 10$, where $M = [(k+1)(n_1+1)/(n_1+n_2+2)]$ is the mode of the distribution and $[s]$ denotes the integer portion of s . Used in conjunction with a direct inverse transformation for other parameter values, the algorithm provides an exact method valid for any parameter values that is numerically more stable than existing methods.

Algorithm *H2PE* is similar to *B2PE* for beta variate generation (Schmeiser and Babu (1980)) and *G2PE* for gamma variate generation (Schmeiser and Lal (1980)), although the details differ substantially due to the discreteness of the hypergeometric distribution. Variates are generated via the acceptance/rejection concept based on the rescaled hypergeometric probability mass function

$$f(x) = \frac{M!(k-M)!(n_1-M)!(n_2-k+M)!}{y!(k-y)!(n_1-y)!(n_2-k+y)!}$$

$$= 0 \quad \text{elsewhere} \quad (1)$$

where $\max(0, k - n_2) - 0.5 \leq x \leq \min(n_1, k) + 0.5$ and $y = [x + 0.5]$. The advantage of this rescaling is discussed in Schmeiser and Kachitvichyanukul (1981).

As shown in Figure 1, the majorizing function is

$$\begin{aligned}
 t(x) &= k_L \exp \{ -\lambda_L (x_L - x - 0.5) \} & -\infty < x \leq x_L - 0.5 \\
 &= 1.0 & x_L - 0.5 < x \leq x_R - 0.5 \\
 &= k_R \exp \{ -\lambda_R (x + 0.5 - x_R) \} & x_R - 0.5 < x < \infty
 \end{aligned} \quad (2)$$

The constants k_L , k_R , λ_L , λ_R , x_L and x_R are defined in Step 0 of the algorithm as function of the parameters n_1 , n_2 and k .

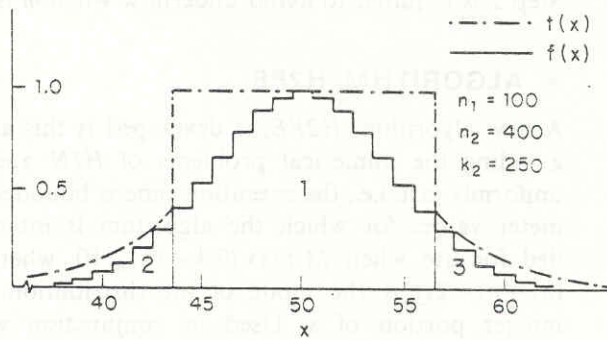


FIGURE 1

Algorithm H2PE

Step 0. (Set-up constants as function of n_1^* , n_2^* , and k^* .

Execute whenever parameter changes.)

0.0 (Determine parameters set for more efficient generation).

$n \leftarrow n_1^* + n_2^*$, $n_1 \leftarrow n_1^*$, $n_2 \leftarrow n_2^*$

If $n_1^* > n_2^*$, set $n_1 \leftarrow n_2^*$, $n_2 \leftarrow n_1^*$

$k \leftarrow k^*$

If $k^* > n/2$, set $k \leftarrow n - k^*$

0.1 (Set-up constants.)

$M \leftarrow \lfloor (k+1)(n_1+1)/(n+2) \rfloor$

If $(M - \max(0, k - n_2) < 10)$, generate x via algorithm HIN and return

$A \leftarrow \ln M! + \ln(n_1 - M)! + \ln(k - M)! + \ln(n_2 - k + M)!$

$D \leftarrow \{ 1.5 \{ (n - k)kn_1n_2 / (n - 1)n^2 \}^{1/2} \} + 0.5$

$$\begin{aligned}
 x_L &\leftarrow M - D + 0.5, \quad x_R \leftarrow M + D + 0.5 \\
 k_L &\leftarrow \exp \{A - \ln x_L! - \ln(n_1 - x_L)! \\
 &\quad - \ln(k - x_L)! - \ln(n_2 - k + x_L)!\} \\
 k_R &\leftarrow \exp \{A - \ln(x_R - 1)! - \ln(n_1 - x_R + 1)! \\
 &\quad - \ln(k - x_R + 1)! - \ln(n_2 - k + x_R - 1)!\} \\
 \lambda_L &\leftarrow -\ln \{x_L(n_2 - k + x_L)/((n_1 - x_L + 1)(k - x_L + 1))\} \\
 \lambda_R &\leftarrow -\ln \{(n_1 - x_R + 1)(k - x_R + 1)/(x_R(n_2 - k + x_R))\} \\
 p_1 &\leftarrow 2D, \quad p_2 \leftarrow k_L/\lambda_L, \quad p_3 \leftarrow p_2 + k_R/\lambda_R
 \end{aligned}$$

Step 1. (Begin logic to generate one hypergeometric variate y . Generate $u \sim U(0, p_3)$ for selecting the region, $v \sim U(0, 1)$ for the accept/reject decision. If region 1 is selected, generate a uniform variate between x_L and x_R .)

Generate $u \sim U(0, p_3)$, $v \sim U(0, 1)$
 If $u > p_1$, go to 2
 Otherwise $y \leftarrow \lfloor x_L + u \rfloor$ and go to 4

Step 2. (Region 2, left exponential tail.)

If $u > p_2$, go to 3
 $y \leftarrow \lfloor x_L + \ln(v)/\lambda_L \rfloor$
 If $y < \max(0, k - n_2)$, go to 1
 $v \leftarrow v(u - p_1)\lambda_L$, go to 4

Step 3. (Region 3, right exponential tail.)

$y \leftarrow \lfloor x_R - \ln(v)/\lambda_R \rfloor$
 If $y > \min(n_1, k)$, go to 1
 $v \leftarrow v(u - p_2)\lambda_R$

Step 4. Acceptance/Rejection Comparison

4.0 (Test for appropriate method of evaluating $f(y)$.)

If $M \geq 100$ and $y > 50$, go to 4.2

4.1 (Evaluate $f(y)$ via recursive relationship

$f(y+1) = f(y) \frac{(n_1 - y)(k - y)}{(y+1)(n_2 - k + y + 1)}$. Start the search from the mode.)

$f \leftarrow 1.0$

If $M < y$

```

then set  $i \leftarrow M$  and
Repeat
   $i \leftarrow i + 1$ 
   $f \leftarrow f(n_1 - i + 1)(k - i + 1)/(i(n_2 - k + i))$ 
Until  $i = y$ 
Otherwise
  If  $M > y$ 
    then set  $i \leftarrow y$  and
    Repeat
       $i \leftarrow i + 1$ 
       $f \leftarrow f i(n_2 - k + i)/((n_1 - i)(k - i))$ 
    Until  $i = M$ 
  End if
End if
If  $v > f$ , go to 1
Otherwise go to 5
4.2 (Squeezing. Check the value of  $\ln(v)$  against upper and
lower bound of  $\ln f(y)$ .)
 $x \leftarrow y, y_1 \leftarrow y + 1, y_M \leftarrow y - M$ 
 $y_n \leftarrow n_1 - y + 1, y_k \leftarrow k - y + 1, n_k \leftarrow n_2 - k + y_1$ 
 $R \leftarrow -y_M/y_1, S \leftarrow y_M/y_n, T \leftarrow y_M/y_k$ 
 $E \leftarrow -y_M/n_k, G \leftarrow y_n y_k/(y_1 n_k) - 1$ 
 $D_G \leftarrow 1$ 
If  $G < 0$ , set  $D_G \leftarrow 1.0 + G$ 
 $G_U \leftarrow G(1.0 + G(-0.5 + G/3.0))$ 
 $G_L \leftarrow G_U - G^4/4D_G$ 
 $X_M \leftarrow M + 0.5, X_n \leftarrow n_1 - M + 0.5, X_k \leftarrow k - M + 0.5$ 
 $n_M \leftarrow n_2 - k + X_M$ 
 $U_b \leftarrow X_M R(1.0 + R(-0.5 + R/3.0))$ 
 $\quad + X_n S(1.0 + S(-0.5 + S/3.0))$ 
 $\quad + X_k T(1.0 + T(-0.5 + T/3.0))$ 
 $\quad + n_M E(1.0 + E(-0.5 + E/3.0)) + y G_U - M G_L + 0.0034$ 
 $A_v \leftarrow \ln(v)$ 
If  $A_v > U_b$ , go to 1
 $D_R \leftarrow X_M R^4$ 
If  $R < 0$ ,  $D_R \leftarrow D_R/(1 + R)$ 
 $D_S \leftarrow X_n S^4$ 
If  $S < 0$ ,  $D_S \leftarrow D_S/(1 + S)$ 
 $D_T \leftarrow X_k T^4$ 

```

If $T < 0$, $D_T \leftarrow D_T / (1 + T)$

$D_E \leftarrow n_M E^4$

If $E < 0$, $D_E \leftarrow D_E / (1 + E)$

If $A_e < U_b - 0.25(D_R + D_S + D_T + D_E) + (y + M)(G_L - G_U) - 0.0078$, go to 5

4.3 (Final Acceptance/Rejection Test.)

If $A_e > A - \ln y! - \ln(n_1 - y)! - \ln(k - y)! - \ln(n_2 - k + y)!$, go to 1

Step 5. (Return appropriate random variate.)

If $k^* < n/2$, go to 5.1

If $n_1^* \leq n_2^*$, go to 5.2

return $y \leftarrow k^* - n_2^* + y$

5.1 If $n_1^* \leq n_2^*$, return y

Otherwise return $y \leftarrow k^* - y$

5.2 Return $y \leftarrow n_1^* - y$

Remark 1 The name *H2PE* means *Hypergeometric, 2 Points, Exponential*, which is analogous to *B2PE* and *G2PE*. The two points are x_L and x_R , at which $f(x)$ must be evaluated in the set-up, Step 0.

Remark 2 The acceptance/rejection logic is implemented via composition, as illustrated in Figure 1. Three subdensities are used, the first is uniform and corresponds to Region 1, and the next two are exponential and correspond to Regions 2 and 3. In Step 0, p_1 , p_2 , and p_3 are the cumulative values used to randomly select the region to be used in each iteration. The probability of selecting each region is proportional to its area.

Remark 3 The expected number of iterations for *H2PE* is:

$$\begin{aligned} \int_{-\infty}^{\infty} t(x) dx / \int_{-\infty}^{\infty} f(x) dx &= p_3 / \left[\frac{1}{f_H(M)} \int_{-\infty}^{\infty} f_H(x) dx \right] \\ &= p_3 \binom{n_1}{M} \binom{n_2}{k-M} / \binom{n_1+n_2}{k} \end{aligned}$$

where $p_3 = \int_{-\infty}^{\infty} t(x) dx$ and M is the mode as defined in Step 0.

Remark 4 The expected number of $U(0, 1)$ values required per a hypergeometric variate is

$$2p_3 \binom{n_1}{M} \binom{n_2}{k-M} / \binom{n_1+n_2}{k}$$

which is obtained by multiplying the expected number of iterations by the two $U(0, 1)$ variates required per iteration.

Remark 5 Logic for the exponential tails parallel to those of *PTPE* and *BTPE* in Schmeiser and Kachitvichyanukul (1981) and Kachitvichyanukul (1982).

Remark 6 Steps 4.0, 4.1, and 4.2 are not required for the validity of the algorithm. They are included to speed up the algorithm by avoiding the time consuming evaluation of the logarithm functions. Propositions 2 and 3 in the Appendix support the use of these one-sided approximations. In general, omitting these steps results in a more compact but slightly slower algorithm. For computers with floating point hardware which include logarithm and other transcendental functions, the omission should not greatly affect the performance of the algorithm.

4. COMPUTATION RESULTS

Algorithm, *H2PE* developed in Section 3 along with *HALIAS* are compared here with the two existing hypergeometric algorithms *HIN* and *HBU*. All four algorithms are implemented in FORTRAN using the VS FORTRAN compiler on the IBM 3033 computer at The University of Iowa. The $U(0, 1)$ numbers are generated using the function GGUBFS(0) from the IMSL library. Algorithms *HIN*, *HBU*, and *HALIAS* have been improved by selecting appropriate set of parameters as discussed in Section 2. Algorithm *H2PE* was submitted for several goodness-of-fit tests on the IBM 3033.

For each set of parameters and algorithm, four replications of 1000 variates were timed. The execution times shown in Table I are the average of the replication averages and the time unit is in millisecond. As expected, the algorithm *HALIAS* has marginal execution time which is at least twice as fast as the algorithms tested.

TABLE I
Comparison of hypergeometric algorithms
(Time in milliseconds)

n_1	n_2	k	<i>H2PE</i>	<i>HIN</i>	<i>HB</i>	<i>HALIAS</i>
20	20	20	0.67	1.32	3.00	0.27
100	100	20	0.73	1.32	2.99	0.27
1000	1000	20	0.73	1.34	3.02	0.27
10000	100	1000	0.80	1.31	†	0.28
100	100000	10000	0.80	1.30	†	0.28
1000	10000	1000	1.36	10.09	†	0.34
10000	1000	1000	1.38	10.07	†	0.33
1000	10000	10000	1.36	10.08	†	0.33
1000	100000	10000	1.41	10.82	†	0.33
10000	100	10000	1.39	10.08	†	0.33
Memory in words			2361	771	390	$1199 + 6m^\ddagger$
Lines of code			117	43	20	71
Setup time			3.75	1.61	0	$a + bm^\S$

†Time proportional to $\min(k, n_1)$.

‡ $m = \min(k, n_1) - \max(0, k - n_2) + 1$.

§ $a = 1.9279$, $b = 0.18039$ on the IBM 3033.

For the parameter sets where algorithm *H2PE* is applicable, ($M - \max(0, k - n_2) \geq 10$), algorithm *H2PE* is at least twice as fast as either *HIN* or *HBU*. For parameter values with large mean and variance, *H2PE* dominates both *HIN* and *HBU* since they are not uniformly fast algorithms.

The set-up times in milliseconds for *H2PE*, *HIN*, and *HBU* are 3.75, 1.61, and 0 respectively. Algorithm *HALIAS* has the setup time that is linear in m ($1.9279 + 0.180393m$ on the IBM 3033). Algorithm *H2PE* also requires more lines of code but requires less memory than *H2PE* when m is large.

5. SUMMARY

The paper presents a comprehensive study of algorithms for generating random variates from the hypergeometric distribution. Of the

two existing algorithms, *HIN* and *HBU*, *HIN* is much preferred over *HBU* in terms of execution speed but develops numerical problems when m is only moderately large. The aliasing algorithm *HALIAS* has the fastest marginal execution time but has the setup time and memory requirement proportional to m . Furthermore, the same numerical problems occur in the setup step when m is moderately large. For *H2PE*, the marginal execution time is faster than *HIN* and *HBU* but is about two to three times slower than *HALIAS*. The setup time is bounded and the memory requirement is constant.

For applications where correlation induction is desired, inversion algorithm implemented with better search technique is always preferred over other algorithms. Reader can refer to Bratley, Fox, and Schrage (1983) for FORTRAN implementation of the bucket-binary search and the index search techniques.

Acknowledgement

This research was supported by the Office of Naval Research contract N00014-79-C-0832. The authors also wish to thank the referees for their comments.

References

- Abramowitz, M. and Stegun, I. A. (1964). *Handbook of Mathematical Functions*. National Bureau of Standards, Applied Mathematics Series, 55.
- Ahrens, J. H. and Kohrt, K.D. (1981). Computer methods for efficient sampling from largely arbitrary statistical distributions. *Computing* 26, 19-31.
- Bratley, P., Fox, B. L. and Schrage, L. E. (1983). *A Guide to Simulation*. Springer-Verlag, New York.
- Fishman, G. S. (1973). *Concepts and Methods in Discrete Event Digital Simulation*. John Wiley and Sons, New York.
- Fishman, G. S. (1978). *Principles of Discrete Event Simulation*. John Wiley and Sons, New York.
- Fox, B. L. (1978). Data structure and computer science techniques in operations research. *Operations Research* 26, 686-717.
- Kachitvichyanukul, V. (1982). Computer Generation of Poisson, Binomial, and Hypergeometric Random Variates, Ph.D. Dissertation, School of Industrial Engineering, Purdue University.
- Kronmal, R. A. and Peterson, A. V. (1979). On the Alias method for generating random variables from a discrete distribution. *American Statistician* 33, 214-218.
- Lieberman, G. J. and Owen, D. B. (1961). *Tables of the Hypergeometric Probability Distribution*. Stanford University Press, Stanford, California.

- McGrath, E. J. and Irving, D. C. (1973). *Techniques for Efficient Monte Carlo Simulation: Volume II; Random Number Generation for Selected Probability Distributions*. National Technical Information Service, U.S. Department of Commerce.
- Schmeiser, B. W. (1980). Random variate generation: A state-of-the-art survey. *Proceedings of the Winter Simulation Conference*, Orlando, Florida, 79-104.
- Schmeiser, B. W. and Babu, A. J. G. (1980). Beta variate generation via exponential majorizing functions. *Operations Research* **28**, 4, 917-926.
- Schmeiser, B. W. and Lal, R. (1980). Squeeze methods for generating gamma variates. *Journal of the American Statistical Association* **75**, 371, 679-682.
- Schmeiser, B. W. and Kachitvichyanukul, V. (1981). Poisson random variate generation. Technical report 81-4, School of Industrial Engineering, Purdue University.
- Walker, A. J. (1977). An efficient method for generating discrete random variables with general distributions. *ACM Transactions on Mathematical Software* **3**, 253-256.

Appendix I

VALIDITY OF H2PE

This appendix discusses the validity of algorithm H2PE. Proposition 1 states that the set-up used in algorithm H2PE is valid, while Propositions 2 and 3 validate the squeeze functions used in the algorithm.

PROPOSITION 1 For $kn_1/(n_1+n_2) \geq 10$ and for all $x \in (-\infty, \infty)$, $t(x) \geq f(x)$, where $f(x)$ and $t(x)$ are defined in Eqs. 1 and 2. Specific constants are defined in Step 0 of algorithm H2PE.

Proof The proof is trivial for $x \in (-\infty, \max(0, k-n_2)-0.5)$ and for $x \in (\min(k, n_1)+0.5, +\infty)$ since $t(x) \geq 0$ and $f(x) = 0$ for all x .

For $x \in [x_L-0.5, x_R-0.5]$, $t(x) = 1$ and $f(x) \leq 1$, hence $t(x) \geq f(x)$.

For $x \in (\max(0, k-n_2)-0.5, x_L-0.5)$, the proof is as follows.

From the set-up in Step 0 of algorithm H2PE, $0 \leq x_L \leq M$. For a fixed set of n_1 , n_2 and k , $y = [x+0.5]$ and $y < x_L$, the following four inequalities are true.

$$x_L^{(x_L-y)} \geq x_L(x_L-1) \dots (y+1) \quad (A.1)$$

$$(n_2 - k + x_L)^{(x_L - y)} \geq (n_2 - k + x_L)(n_2 - k + x_L - 1) \dots (n_2 - k + y + 1) \quad (\text{A.2})$$

$$(n_1 - x_L + 1)^{(x_L - y)} \leq (n_1 - y)(n_1 - y + 1) \dots (n_1 - x_L + 1) \quad (\text{A.3})$$

$$(k - x_L + 1)^{(x_L - y)} \leq (k - y)(k - y + 1) \dots (k - x_L + 1) \quad (\text{A.4})$$

The first two inequalities yield

$$\{x_L(n_2 - k + x_L)\}^{(x_L - y)} \geq \frac{x_L!(n_2 - k + x_L)!}{y!(n_2 - k + y)!} \quad (\text{A.5})$$

Combining inequalities A.3 and A.4 yields

$$\{(n_1 - x_L + 1)(k - x_L + 1)\}^{x_L - y} \leq \frac{(n_1 - y)!(k - y)!}{(n_1 - x_L)!(k - y_L)!}$$

or its reciprocal

$$\left\{ \frac{1}{(n_1 - x_L + 1)(k - x_L + 1)} \right\}^{x_L - y} \geq \frac{(n_1 - x_L)!(k - x_L)!}{(n_1 - y)!(k - y)!} \quad (\text{A.6})$$

Combining inequalities A.5 and A.6 yields

$$\left\{ \frac{x_L(n_2 - k + x_L)}{(n_1 - x_L + 1)(k - x_L + 1)} \right\}^{x_L - y} \geq \frac{x_L!(n_2 - k + x_L)!(n_1 - x_L)!(k - x_L)!}{y!(n_2 - k + y)!(n_1 - y)!(k - y)!}$$

Multiplying both sides by $n_1!$ and $n_2!$, dividing both sides by $\binom{n_1 + n_2}{k}$ and rearranging yields

$$\frac{\binom{n_1}{x_L} \binom{n_2}{k - x_L}}{\binom{n_1 + n_2}{k}} \left\{ \frac{x_L(n_2 - k + x_L)}{(n_1 - x_L + 1)(k - x_L + 1)} \right\}^{x_L - y} \geq \frac{\binom{n_1}{y} \binom{n_2}{k - y}}{\binom{n_1 + n_2}{k}}$$

Dividing both sides by the hypergeometric probability evaluated at the mode M leads to the desired result; i.e.,

$$t(x) \geq f(x)$$

Similar logic leads to the proof for all $x \in (x_R - 0.5, \min(n_1, k) + 0.5)$.

Lemma 1 is needed for the proofs of Propositions 2 and 3.

LEMMA 1 Let $\delta(n_1, n_2, k, M, y)$ be defined as

$$\begin{aligned} & [(M+1)^{-1} + (k-M+1)^{-1} + (n_1-M+1)^{-1} + (n_2-k+M+1)^{-1} \\ & - (y+1)^{-1} - (k-y+1)^{-1} - (n_1-y+1)^{-1} - (n_2-k+y+1)^{-1}] / 12. \\ & - [(M+1)^{-3} + (k-M+1)^{-3} + (n_1-M+1)^{-3} + (n_2-k+M+1)^{-3} \\ & - (y+1)^{-3} - (k-y+1)^{-3} - (n_1-y+1)^{-3} - (n_2-k+y \\ & + 1)^{-3}] / 360.0 + \dots \end{aligned}$$

Then for

$$n_1 \geq n'_1, n_2 \geq n'_2, k \geq k', M \geq M', \text{ and } y \geq y',$$

$$\delta_L(n'_1, n'_2, k', M', y') \leq \delta(n_1, n_2, k, M, y) \leq \delta_U(n'_1, n'_2, k', M', y')$$

where

$$\begin{aligned} \delta_L(n'_1, n'_2, k', M', y') = & [-(y'+1)^{-1} - (k'-y'+1)^{-1} - (n'_1-y'+1)^{-1} \\ & - (n'_2-k'+y'+1)^{-1}] / 12 - [(M'+1)^{-3} + (k'-M'+1)^{-3} \\ & + (n'_1-M'+1)^{-3} + (n'_2-k'+M'+1)^{-3}] / 360.0 + \dots \end{aligned}$$

and

$$\begin{aligned} \delta_U(n'_1, n'_2, k', M', y') = & [(M'+1)^{-1} + (k'-M'+1)^{-1} + (n'_1-M'+1)^{-1} \\ & + (n'_2-k'+M'+1)^{-1}] / 12.0 + [(y'+1)^{-3} + (k'-y'+1)^{-3} \\ & + (n'_1-M'+y')^{-3} + (n'_2-k'+y'+1)^{-3}] / 360.0 + \dots \end{aligned}$$

Proof $\delta_L(n'_1, n'_2, k', M', y')$ can be obtained by minimizing $\delta(n_1, n_2, k, M, y)$ term by term and $\delta_U(n'_1, n'_2, k', M', y')$ by maximizing term by term. ||

PROPOSITION 2 For $n_1 \geq n'_1, n_2 \geq n'_2, k \geq k', M \geq M'$, and $y \geq y'$,

$$\begin{aligned} U_b = & (M + 0.5)R(1.0 + R(-0.5 + R/3)) \\ & + (n_1 - M + 0.5)S(1.0 + S(-0.5 + S/3)) \\ & + (k - M + 0.5)T(1.0 + T(-0.5 + T/3)) \\ & + (n_2 - k + M + 0.5)E(1.0 + E(-0.5 + E/3)) \\ & + yG(1.0 + G(-0.5 + G/3)) - MG(1.0 + G(-0.5 + G(1/3 - \rho G/4))) \\ & + \delta_U(n'_1, n'_2, k', M', y') \geq \ln f(y). \end{aligned}$$

where

$$\begin{aligned} R &= (M + 1)/(y + 1) - 1, & S &= (n_1 - M + 1)/(n_1 - y + 1) - 1 \\ T &= (k - M + 1)/(k - y + 1) - 1, & E &= (n_2 - k + M + 1)/(n_2 - k + y + 1) - 1 \\ G &= (n_1 - y + 1)(k - y + 1)/(y + 1)(n_2 - k + y + 1) - 1 \end{aligned}$$

and

$$\rho = \begin{cases} 1 & \text{if } G \geq 0 \\ (1 + G)^{-1} & \text{if } G < 0 \end{cases}$$

Proof Taking logarithms on both sides of $f(y)$ yields

$$\begin{aligned} \ln f(y) = & \ln M! + \ln(n_1 - M)! + \ln(k - M)! + \ln(n_2 - k + m)! \\ & - \ln y! - \ln(n_1 - y)! - \ln(k - y)! - \ln(n_2 - k + y)! \end{aligned}$$

Direct substitution of Stirling's formula for the logarithm of factorial and simplifying yields the expression

$$\begin{aligned} \ln f(y) = & (M + 0.5) \ln [(M + 1)/(y + 1)] \\ & + (n_1 - M + 0.5) \ln [(n_1 - M + 1)/(n_1 - y + 1)] \\ & + (k - M + 0.5) \ln \{(k - M + 1)/(k - y + 1)\} \end{aligned}$$

$$\begin{aligned}
& + (n_2 - k + M + 0.5) \ln [(n_2 - k + M + 1)/(n_2 - k + y + 1)] \\
& + y \ln [(n_1 - y + 1)(k - y + 1)/(y + 1)(n_2 - k + y + 1)] \\
& - M \ln \{(n_1 - y + 1)(k - y + 1)/(y + 1)(n_2 - k + y + 1)\} \\
& + \delta(n_1, n_2, k, M, y)
\end{aligned}$$

Applying Result 1 to the first five terms, Result 2 to the sixth and Lemma 1 to the last term yields the result. ||

PROPOSITION 3 For $n_1 \geq n'_1$, $n_2 \geq n'_2$, $k \geq k'$, $M \geq M'$, and $y \geq y'$,

$$\begin{aligned}
U_L &= U_b - D_r - D_s - D_t - D_e + (y + M)(G_L - G_u) \\
& - \delta_U(n'_1, n'_2, k', M', y') \\
& + \delta_L(n'_1, n'_2, k', M', y') \leq \ln f(y)
\end{aligned}$$

where

$$\begin{aligned}
D_r &= (M + 0.5)R^4\Delta/45\Delta = \begin{cases} 1 & \text{if } R \geq 0 \\ (1 + R)^{-1} & \text{if } R < 0 \end{cases} \\
D_s &= (n_1 - M + 0.5)S^4\theta/45\theta = \begin{cases} 1 & \text{if } S \geq 0 \\ (1 + S)^{-1} & \text{if } S < 0 \end{cases} \\
D_t &= (k + M + 0.5)T^4\phi/45\phi = \begin{cases} 1 & \text{if } T \geq 0 \\ (1 + T)^{-1} & \text{if } T < 0 \end{cases} \\
D_e &= (n_2 - k + M0.5)E^4\omega/45\omega = \begin{cases} 1 & \text{if } E \geq 0 \\ (1 + E)^{-1} & \text{if } E < 0 \end{cases}
\end{aligned}$$

$$G_U = G(1.0 + G(-0.5 + G/3))$$

$$G_L = G_U - G^4\rho/45\rho = \begin{cases} 1 & \text{if } G \geq 0 \\ (1 + G)^{-1} & \text{if } G < 0 \end{cases}$$

and R , S , T , E , and G are defined as in Proposition 2.

Proof From Proposition 2

$$\begin{aligned}
 \ln f(y) = & (M + 0.5) \ln [(M + 1)/(y + 1)] \\
 & + (n_1 - M + 0.5) \ln [(n_1 - M + 1)/(n_1 - y + 1)] \\
 & + (k - M + 0.5) \ln [(k - M + 1)/(k - y + 1)] \\
 & + (n_2 - k + M + 0.5) \ln [(n_2 - k + M + 1)/(n_2 - k + y + 1)] \\
 & + y \ln \{(n_1 - y + 1)(k - y + 1)/(y + 1)(n_2 - k + y + 1)\} \\
 & - M \ln \{(n_1 - y + 1)(k - y + 1)/(y + 1)(n_2 - k + y + 1)\} \\
 & + \delta(n_1, n_2, k, M, y)
 \end{aligned}$$

The expression for U_L is obtained by applying Result 2 to the first five terms of $\ln f(y)$, Result 1 to the sixth term and Lemma 1 to the last term, which yields

$$\begin{aligned}
 U_L = & (M + 0.5)R(1 + R(-0.5 + R(1/3 - R\Delta/4))) \\
 & + (n_1 - M + 0.5)S(1 + S(-0.5 + S(1/3 - S\theta/4))) \\
 & + (k - M + 0.5)T(1 + T(-0.5 + T(1/3 - T\phi/4))) \\
 & + (n_2 - k + M + 0.5)E(1 + E(-0.5 + E(1/3 - E\omega/4))) \\
 & + yG(1 + G(-0.5 + G(1/3 - G\rho/4))) \\
 & - MG(1 + G(-0.5 + G/3)) \\
 & + \delta_L(n'_1, n'_2, k', M', y')
 \end{aligned}$$

Rearranging the right hand side of the expression and rewriting it in terms of U_b yields the result. ||

The values of n'_1 and n'_2 used in $H2PE$ are 200 whereas k' is 199 and y' is 50, which implies that M' is 100.